GenAI Documentation

Name: Emerson Sanchez

If GenAI was used (keep each response as brief as possible):

Date Used: March 23rd

Tool Disclosure: ChatGPT 5.2

Purpose of Use: GenAI was used to fix the errors that were causing issues in the project as well as provide the coded needed for the debug menu.

Summary of Interaction: Provided solutions and additional code for the debug menu.

Human Decision Point(s): I decided to take the generated AI code, looked at the course material and did a second check to ensure that the code worked as intended for this sidequest.

Integrity & Verification Note: Provided GenAI the code and referenced lecture to confirm it works with JS and JSON files.

Scope of GenAI Use: GenAI contributed to refining and finding sections of the code to make it work.

Limitations or Misfires: GenAI kept adding new pieces of code and sometimes kept repeating answers.

Summary of Process (Human + Tool)

- Used the code provided by GenAI, ensured that the code was stable and did not have issues that would prevent the game from being in an unstable state.

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Took code directly from AI, and used some of its suggestions to refine the code created for other sections of this sidequest.

Old Code

```
C: > Users > emers > OneDrive > Desktop > GBDA Folder > GBDA 302 > Example Files > SQW9 > updated > Week9_Example03 > JS sketch.js >
debugpan                Aa ab .*  No results
function startMusicIfNeeded() {
  const maybePromise = userStartAudio();
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}

function keyPressed() {
  startMusicIfNeeded();
}

function mousePressed() {
  startMusicIfNeeded();
}

function touchStarted() {
  startMusicIfNeeded();
  return false;
}

function draw() {
  // --- BACKGROUND ---
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  // --- PLAYER CONTROLS ---
  // first check to see if the player is on the ground
  let grounded = sensor.overlapping(ground);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play(); // plays once to end
```

New/added Code

```
JS sketch.js > drawDebugPanel
194  function debugRect(x, y, w, h, col) {
202  }
203  function drawHitboxes() {
204    debugRect(player.x, player.y, player.w, player.h, color(255, 80, 80));
205    debugRect(sensor.x, sensor.y, sensor.w, sensor.h, color(80, 180, 255));
206    for (let t of groundDeep) debugRect(t.x, t.y, t.w, t.h, color(80, 255, 80));
207    for (let t of groundDeep) debugRect(t.x, t.y, t.w, t.h, color(160, 120, 60));
208  }
209  function drawDebugPanel(grounded) {
210    camera.off();
211
212    push();
213    fill(0, 150); // semi-transparent background
214    noStroke();
215    rect(10, 10, 140, 80);
216        You, 3 minutes ago • Uncommitted changes
217    fill(255);
218    textSize(10);
219    textAlign(LEFT, TOP);
220
221    text("DEBUG PANEL", 15, 15);
222    text("Grounded: " + grounded, 15, 30);
223    text("Moon Gravity: " + debug.moonGravity, 15, 45);
224    text("Hitboxes: " + debug.showHitboxes, 15, 60);
225
226    pop();
227
228    camera.on();
229  }
230  function drawDebugHint() {
231    camera.off();
232    camera.on();
233  }
234
235  function draw() {
236    // --- BACKGROUND ---
237    camera.off();
238    imageMode(CORNER);
239    image(bgImg, 0, 0, bgImg.width, bgImg.height);
240    camera.on();
241
```

debugpan Aa ab .* ? of 2

let groundDeep: any

Appendix

https://chatgpt.com/share/69c1f7d3-15e4-8004-b30e-8b1dae9d1735

```
  Week 9 — Example 3: Adding Sound & Music

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Mar. 19, 2026

  Controls:
    A or D (Left / Right Arrow)   Horizontal movement
    W (Up Arrow)                  Jump
    Space Bar                     Attack

Debug controls:
\ --> debug pannel
G --> Moon gravity
H --> Hitbox visuals

  Tile key:
    g = groundTile.png       (surface ground)
    d = groundTileDeep.png   (deep ground, below surface)
      = empty (no sprite)
*/

let player;
let playerImg, bgImg;
let jumpSfx, musicSfx;
let musicStarted = false;

let playerAnis = {
  idle: { row: 0, frames: 4, frameDelay: 10 },
  run: { row: 1, frames: 4, frameDelay: 3 },
  jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 },
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

let ground, groundDeep;
let groundImg, groundDeepImg;

let attacking = false; // track if the player is attacking
let attackFrameCounter = 0; // tracking attack animation

let debug = {
  open: false,
```

```
  moonGravity: false,
  showHitboxes: false,
};

const GRAVITY_NORMAL = 10;
const GRAVITY_MOON = 1.6;

// --- TILE MAP ---
// an array that uses the tile key to create the level
let level = [
  "              ",
  "              ",
  "              ",
  "              ",
  "              ",
  "       ggg    ",
  "gggggggggggggg", // surface ground
  "dddddddddddddd", // deep ground
];

// --- LEVEL CONSTANTS ---
// camera view size
const VIEWW = 320,
  VIEWH = 180;

// tile width & height
const TILE_W = 24,
  TILE_H = 24;

// size of individual animation frames
const FRAME_W = 32,
  FRAME_H = 32;

// Y-coordinate of player start (4 tiles above the bottom)
const MAP_START_Y = VIEWH - TILE_H * 4;

// gravity
const GRAVITY = 10;

function preload() {
  // --- IMAGES ---
  playerImg = loadImage("assets/foxSpriteSheet.png");
  bgImg = loadImage("assets/combinedBackground.png");
  groundImg = loadImage("assets/groundTile.png");
```

```
  groundDeepImg = loadImage("assets/groundTileDeep.png");

  // --- SOUND ---
  if (typeof loadSound === "function") {
    jumpSfx = loadSound("assets/sfx/jump.wav");
    musicSfx = loadSound("assets/sfx/music.wav");
  }
}

function setup() {
  // pixelated rendering with autoscaling
  new Canvas(VIEWW, VIEWH, "pixelated");

  // needed to correct an visual artifacts from attempted antialiasing
  allSprites.pixelPerfect = true;

  world.gravity.y = GRAVITY;

  // Try to start background music immediately.
  if (musicSfx) musicSfx.setLoop(true);
  startMusicIfNeeded();

  // --- TILE GROUPS ---
  ground = new Group();
  ground.physics = "static";
  ground.img = groundImg;
  ground.tile = "g";

  groundDeep = new Group();
  groundDeep.physics = "static";
  groundDeep.img = groundDeepImg;
  groundDeep.tile = "d";

  // a Tiles object creates a level based on the level map array (defined at the beginning)
  new Tiles(level, 0, 0, TILE_W, TILE_H);

  // --- PLAYER ---
  player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player
  player.spriteSheet = playerImg; // use the sprite sheet
  player.rotationLock = true; // turn off rotations (player shouldn't rotate)

  // player animation parameters
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
```

```
  player.anis.offset.y = -4; // offset the collision box up
  player.addAnis(playerAnis); // add the player animations defined earlier
  player.ani = "idle"; // default to the idle animation
  player.w = 18; // set the width of the collsion box
  player.h = 20; // set the height of the collsion box
  player.friction = 0; // set the friciton to 0 so we don't stick to walls
  player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce

  // --- GROUND SENSOR --- for use when detecting if the player is standing on the ground
  sensor = new Sprite();
  sensor.x = player.x;
  sensor.y = player.y + player.h / 2;
  sensor.w = player.w;
  sensor.h = 2;
  sensor.mass = 0.01;
  sensor.removeColliders();
  sensor.visible = false;
  let sensorJoint = new GlueJoint(player, sensor);
  sensorJoint.visible = false;
}

function startMusicIfNeeded() {
  if (musicStarted || !musicSfx) return;

  const startLoop = () => {
    if (!musicSfx.isPlaying()) musicSfx.play();
    musicStarted = musicSfx.isPlaying();
  };

  // Some browsers require a user gesture before audio can start.
  const maybePromise = userStartAudio();
  if (maybePromise && typeof maybePromise.then === "function") {
    maybePromise.then(startLoop).catch(() => {});
  } else {
    startLoop();
  }
}

function keyPressed() {
  startMusicIfNeeded();
}

if (key === "\"") {
  debug.open = !debug.open;
```

```
}

if (key === "g" || key === "G") {
  debug.moonGravity = !debug.moonGravity;
  world.gravity.y = debug.moonGravity ? GRAVITY_MOON : GRAVITY_NORMAL;
}

if (key === "h" || key === "H") {
  debug.showHitboxes = !debug.showHitboxes;
}

function mousePressed() {
  startMusicIfNeeded();
}

function touchStarted() {
  startMusicIfNeeded();
  return false;
}

function debugRect(x, y, w, h, col) {
  push();
  noFill();
  stroke(col || color(0, 255, 0));
  strokeWeight(0.5);
  rectMode(CENTER);
  rect(x, y, w, h);
  pop();
}
function drawHitboxes() {
  debugRect(player.x, player.y, player.w, player.h, color(255, 80, 80));
  debugRect(sensor.x, sensor.y, sensor.w, sensor.h, color(80, 180, 255));
  for (let t of ground) debugRect(t.x, t.y, t.w, t.h, color(80, 255, 80));
  for (let t of groundDeep) debugRect(t.x, t.y, t.w, t.h, color(160, 120, 60));
}
function drawDebugPanel(grounded) {
  camera.off();
  camera.on();
}
function drawDebugHint() {
  camera.off();
  camera.on();
}
```

```
function draw() {
  // --- BACKGROUND ---
  camera.off();
  imageMode(CORNER);
  image(bgImg, 0, 0, bgImg.width, bgImg.height);
  camera.on();

  // --- PLAYER CONTROLS ---
  // first check to see if the player is on the ground
  let grounded = sensor.overlapping(ground);

  // -- ATTACK INPUT --
  if (grounded && !attacking && kb.presses("space")) {
    attacking = true;
    attackFrameCounter = 0;
    player.vel.x = 0;
    player.ani.frame = 0;
    player.ani = "attack";
    player.ani.play(); // plays once to end
  }

  // -- JUMP --
  if (grounded && kb.presses("up")) {
    player.vel.y = -4;
    if (jumpSfx) jumpSfx.play();
  }

  // --- STATE MACHINE ---
  if (attacking) {
    attackFrameCounter++;
    // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed)
    if (attackFrameCounter > 12) {
      attacking = false;
      attackFrameCounter = 0;
    }
  } else if (!grounded) {
    player.ani = "jump";
    player.ani.frame = player.vel.y < 0 ? 0 : 1;
  } else {
    player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle";
  }

  // --- MOVEMENT ---
  if (!attacking) {
```

```
    player.vel.x = 0;
    if (kb.pressing("left")) {
      player.vel.x = -1.5;
      player.mirror.x = true;
    } else if (kb.pressing("right")) {
      player.vel.x = 1.5;
      player.mirror.x = false;
    }
  }

  // --- KEEP IN VIEW ---
  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);

  if (debug.showHitboxes) drawHitboxes();
  if (debug.open) drawDebugPanel(grounded);
  else drawDebugHint();
}
```

what could the cause of the debug to not work?

The issue isn't with your debug logic itself — it's with where your key handling code is placed.

Right now, this part:

```
if (key === "\'") {
  debug.open = !debug.open;
}

if (key === "g" || key === "G") {
  debug.moonGravity = !debug.moonGravity;
  world.gravity.y = debug.moonGravity ? GRAVITY_MOON : GRAVITY_NORMAL;
}

if (key === "h" || key === "H") {
  debug.showHitboxes = !debug.showHitboxes;
}
```

is outside of any function. That means it only runs once when the script loads — not when a key is pressed.

✅ Fix: Move it inside keyPressed()
You already have a keyPressed() function:

```
function keyPressed() {
  startMusicIfNeeded();
```

}
Just move your debug controls inside it:

```
function keyPressed() {
  startMusicIfNeeded();

  if (key === "`") {
    debug.open = !debug.open;
  }

  if (key === "g" || key === "G") {
    debug.moonGravity = !debug.moonGravity;
    world.gravity.y = debug.moonGravity ? GRAVITY_MOON : GRAVITY_NORMAL;
  }

  if (key === "h" || key === "H") {
    debug.showHitboxes = !debug.showHitboxes;
  }
}
```

⚠️ Small extra gotcha
You wrote:

```
if (key === "\`")
```

You don't need the escape here. Just use:

```
if (key === "`")
```

🧠 Why this broke
In p5.js:

keyPressed() runs every time a key is pressed

Code outside functions runs once at startup

So your debug toggles were never being triggered.

✅ Quick checklist
If it still doesn't work after fixing:

Make sure the canvas is focused (click it once)

Confirm no console errors

Try logging:

console.log(key);
inside keyPressed() to verify inputs

If you want next step: I can help you turn that debug panel into an actual UI (text overlay showing gravity state, grounded, etc.), since your drawDebugPanel() is currently empty.

why is my debug menu not showing anything?

Your debug system is actually toggling correctly now — the reason you don't see anything is much simpler:

👉 Your debug panel doesn't draw anything.

🔍 The problem
This function is empty:

```
function drawDebugPanel(grounded) {
  camera.off();
  camera.on();
}
```

So when this runs:

```
if (debug.open) drawDebugPanel(grounded);
```

…it technically works, but it renders nothing to the screen.

✅ Fix: Actually draw something
You need to add text (or shapes) inside drawDebugPanel().

Here's a simple working version:

```
function drawDebugPanel(grounded) {
  camera.off();

  push();
  fill(0, 150); // semi-transparent background
  noStroke();
  rect(10, 10, 140, 80);

  fill(255);
  textSize(10);
  textAlign(LEFT, TOP);

  text("DEBUG PANEL", 15, 15);
```

```
  text("Grounded: " + grounded, 15, 30);
  text("Moon Gravity: " + debug.moonGravity, 15, 45);
  text("Hitboxes: " + debug.showHitboxes, 15, 60);

  pop();

  camera.on();
}
```

⚠️ Same issue with your hint
This also does nothing:

```
function drawDebugHint() {
  camera.off();
  camera.on();
}
```

So even when debug is closed, there's no hint shown.

You could add:

```
function drawDebugHint() {
  camera.off();

  push();
  fill(255);
  textSize(8);
  text("Press ' to open debug", 10, VIEWH - 10);

  pop();

  camera.on();
}
```

🧠 Key takeaway
Your logic is fine:

debug.open toggles ✅

drawDebugPanel() is called ✅

But:

Rendering = separate step — nothing shows unless you explicitly draw it.

If you want, I can help you make a proper dev-style debug overlay (FPS, velocity, position, collisions) — super useful for platformers.

3 messages remaining. Start a free Plus trial to keep the conversation going

Try Plus free